

Tactix v1.0

System Architecture, Database Specifications & Production Deployment Guide

Generated: August 20, 2026 | Scope: Full Codebase Audit & Database Playbook

1. Executive Codebase Summary

Tactix is a production-grade, gamified chess tactics training and spaced repetition (SRS) platform. It enables users to import PGN tactical files, solve interactive puzzles, analyze tactical strengths across dynamic radar charts, review failed puzzles with the SuperMemo SM-2 algorithm, and earn XP/Coins while building daily streak rewards.

Core Technology Stack:

- **Frontend:** React 18, TypeScript, Vite, Tailwind CSS, Lucide Icons, react-chessboard, chess.js, Recharts (Radar analytics), Web Audio API.
- **Backend API:** Node.js + Express.js supporting JWT Authentication, bcrypt password hashing, Nodemailer password resets, and Prisma ORM cloud synchronization.
- **Database Layer:** PostgreSQL managed via Prisma ORM (prisma/schema.prisma). Includes intelligent in-memory / localStorage fallback if DB connection is offline.
- **State & Storage:** Hybrid IndexedDB / LocalStorage client-side persistence synchronized automatically with PostgreSQL cloud endpoints.

2. Database Configuration & PostgreSQL Details

Tactix uses **PostgreSQL 14+** as its relational database store, configured via Prisma ORM. The server checks for the presence of DATABASE_URL in environment variables. If present, full PostgreSQL cloud sync is enabled.

Database Connection String Format:

```
DATABASE_URL="postgresql://<USER>:<PASSWORD>@<HOST>:<PORT>/<DB_NAME>?schema=public"
```

Recommended Database Names:

- **Local Development DB:** tactix_db
- **Production Cloud DB:** tactix_prod

Database Schema Overview (6 Core Prisma Models):

Model	Table Name	Key Fields & Types	Purpose & Description
User	users	id (UUID), email, username, passwordHash, level, xp, coins, performanceRating, currentStreak	User accounts, auth credentials, password reset tokens, and gamification state.
Collection	collections	id, userId, name, category, description, icon, color, isCompleted	Groups puzzles into custom tactical sets (e.g., Pins, Forks, Endgames).
Puzzle	puzzles	id, collectionId, FEN, sideToMove, solutionMoves[], solutionUCI[], difficulty, rating	Individual tactical puzzle positions with move validation sequences and solve stats.
PuzzleAttempt	puzzle_attempts	id (UUID), userId, puzzleId, solved (Bool), solveTimeMs, mistakes, xpEarned, coinsEarned	Historical attempt log tracking solve times, accuracy, and earned rewards.
SRSState	srs_states	id (UUID), userId, puzzleId, repetitions, easeFactor, intervalDays, dueDate	SuperMemo SM-2 spaced repetition state scheduling due dates for failed puzzles.
UserAchievement	user_achievements	id (UUID), userId, achievementId, unlocked (Bool), currentValue, unlockedAt	Tracks unlock status and progress towards milestones and badge rewards.

3. Step-by-Step Database Setup & Deployment Guide

Option A: Local PostgreSQL Setup (Development)

- **Step 1: Start PostgreSQL Instance**
Option 1 (Docker): `docker run --name tactix-pg -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=tactix_db -p 5432:5432 -d postgres`
Option 2: Install PostgreSQL locally and run `CREATE DATABASE tactix_db; in psql.`
- **Step 2: Configure Environment File (.env)**
Create a .env file in the project root with:
`DATABASE_URL="postgresql://postgres:postgres@localhost:5432/tactix_db?schema=public"`
`PORT=4000`
`JWT_SECRET="tactix-super-secret-key"`
- **Step 3: Push Schema & Generate Client**
Run `npm run prisma:generate` to build client JS.
Run `npx prisma db push` to apply schema to PostgreSQL.
Run `npm run prisma:studio` (optional) to view database tables in browser.

Option B: Production Cloud Deployment

- **Step 1: Provision Cloud PostgreSQL Database**
Create a database on **Neon.tech**, **Supabase**, or **Railway**.
Copy the connection URL (e.g. `postgresql://user:pass@ep-xyz.neon.tech/tactix_prod?sslmode=require`).
- **Step 2: Push Production Database Schema**
Execute `npx prisma db push` with the production `DATABASE_URL` set.
- **Step 3: Deploy Backend Express Server**
Deploy server code to **Render.com** or **Railway.app**.
Build Command: `npm install && npm run prisma:generate`

Start Command: node server/index.js
Set Env Vars: DATABASE_URL, JWT_SECRET, CORS_ORIGIN.

• **Step 4: Deploy Frontend Client Application**
Deploy React app to **Vercel** or **Netlify**.
Build Command: npm run build (Output dir: dist)
Set Env Var: VITE_API_URL="https://your-backend-api.onrender.com".

Target Task	Command Line Execution
Run Full Dev Stack	npm run dev (Runs Express server & Vite client concurrently)
Generate Prisma Client	npm run prisma:generate
Sync DB Schema to Postgres	npx prisma db push
Database GUI Admin Studio	npm run prisma:studio
Build Production Bundle	npm run build

4. Comprehensive Codebase Map & File Inventory

Below is a complete audit of where every component, service, and backend script is located in the codebase:

Category	File Path / Module	Description & Responsibility
Config & DB	prisma/schema.prisma	Prisma ORM schema definitions for User, Collection, Puzzle, Attempt, SRS, Achievement.
Config & DB	package.json	Project scripts, build commands, and full dependency declarations.
Backend API	server/index.js	Express server implementing JWT Auth, password hashing, mailer, and PostgreSQL sync API.
Backend API	server/prismaClient.js	Prisma client singleton instance connecting Node.js with PostgreSQL.
Frontend Core	src/App.tsx	Main application shell, tab routing, sound engine hooks, and cloud sync initializer.
Services	src/services/storage.ts	IndexedDB / LocalStorage manager with automatic backend cloud sync.
Services	src/services/authApi.ts	REST API client for user registration, login, profile check, and JWT storage.
Services	src/services/libraryApi.ts	REST API client synchronizing local puzzle collections to PostgreSQL DB.
Services	src/services/pgnParser.ts	PGN parser extracting chess positions, SAN moves, and UCI validation paths.
Services	src/services/srsEngine.ts	SuperMemo SM-2 algorithm calculating repetition intervals and due dates.
Services	src/services/gamification.ts	XP, level formulas, coin earnings, streaks, rating, and achievement engine.
Services	src/services/soundEngine.ts	Web Audio API synthesizer producing move, capture, check, and victory sound FX.
UI Components	src/components/solver/	Interactive chessboard puzzle solver with move validation, hints, timer, solution reveal.
UI Components	src/components/collections/	Collection manager, PGN import modal, collection cards, and puzzle viewer.

UI Components	src/components/review/	Spaced Repetition review queue and failed puzzle workout mode.
UI Components	src/components/insights/	Tactical strength radar chart (Recharts spider graph) and fatigue analysis.
UI Components	src/components/dashboard/	Homepage stats header, Level/XP bar, daily streak counter, and recent activity.
UI Components	src/components/profile/	User profile summary, unlockable achievement showcase, and settings.
UI Components	src/components/auth/	Authentication modal handling login, sign-up, and password recovery.